End-of-Module Assessment (EMA) — e-Portfolio

Advanced Object-Oriented Design and Programming (AOODP)

Assignment Title: e-Portfolio — Showcase of Artefacts and Professional Development Plan

Module: Advanced Object-Oriented Design and Programming (AOODP)

Course/Programme: [Insert your course / programme]

University ID: 12701559

Submission date: 17th October 2026

Word count: 3,445 words (Tasks 1–4 body text)

Counting rule: prose in the Task 1–4 narrative. Excluded: cover sheet, section headings, the figure and its caption, the reference list and Appendices A–C. Prose-only counting keeps the figure within the stated word budget.

Declaration:

I hereby certify that this is my own original work. I have used any relevant technologies according to the assignment guidelines set out on Moodle.

Signature: 12701559 Date: 17th October 2026

Task 1 – Summarise Key Learning Outcomes

The capstone is a transaction risk-assessment engine: transactions in, governance-instrumented decisions out. Appendix A maps every claim below to a file and a captured run.

1.1 Understand and implement secure coding practices

Security here is structural, not appended. **Construction-time validation:** validation.py centralises every check and domain error, and Transaction is a frozen slotted dataclass whose __post_init__ calls those validators, so an invalid transaction cannot exist; error messages never echo the rejected value, since reflecting untrusted input is the mechanism behind injection defects (OWASP Foundation, 2021). **Boundary minimisation:** RedactingDecorator is outermost, so no inner layer, audit sink included, sees a raw identifier, and ProviderConfig refuses a hosted family with a non-https:// endpoint or redact_before_send false. **Untrusted replies:** ModelBackedScoringStrategy rejects a non-mapping reply and a missing, boolean, non-numeric or out-of-range score, translating vendor exceptions into ProviderUnavailableError.

1.2 Apply advanced object-oriented principles to complex problems

Five concerns normally entangled in one scoring script are separated — feature derivation (feature_store.py), algorithm (strategies.py), cross-cutting policy (decorators.py), lifecycle (model_registry.py) and output format (reporting.py) — and joined only by the composition root in service.py. Encapsulation uses frozen value objects, polymorphism RiskScoringStrategy and ReportVisitor, abstraction the five-line ScoringBackend protocol, and inversion of control constructor injection of transport, clock, context provider, backend and sink. Algorithm, provider, model version and report format are each replaceable without editing a consumer.

1.3 Utilise design patterns for reusable, maintainable, flexible code

Pattern	Where	Flexibility it buys
Strategy	strategies.py	Rules, statistical or model-backed scoring swapped by one config string
Decorator	decorators.py	Redaction, metrics, audit, caching, fallback composed at one assembly point
Visitor	reporting.py	Reports added as classes; ChannelRiskVisitor touched no other file
Abstract Factory	providers.py	A whole provider family swapped together, consistency checked
Dependency Injection	service.py	Every outside-world dependency passed in, making tests deterministic

Each pattern was chosen against a named alternative, and the rejected alternatives are part of the work.

1.4 Design architectures suitable for large-scale systems, ensuring security and robustness

Each layer scales along the axis that matters: feature computation is pure and therefore cacheable and parallelisable; caching is an inner decorator, so it can become distributed without touching governance; the registry centralises lifecycle state, so N scoring instances read one answer to which version is live. Robustness is demonstrated rather than asserted: a provider outage produces three transport attempts, an explicit [degraded: primary unavailable] rationale and a substituted decision that still reaches the audit trail; a hostile payload {"confidence": 0.99, "verdict": "looks fine to me"} is rejected; an illegal archived -> production promotion is refused.

1.5 Develop software adaptable for AI models and efficient for data science tasks

Three properties were designed in, not retrofitted. **Abstraction:** the AI layer is reached only through ScoringBackend, so replacing a hosted LLM with a local model or a fake is a composition-root change. **Contracts:** StrategyOutput carries contributions and rationale, making explainability part of the algorithm contract, and FeatureDefinition is named, versioned and owned, so training and serving reference the same definitions. **Lifecycle:** the registry makes model version a runtime fact, the precondition for shadow deployment and rollback.

1.6 Link to practical work

Learning objective	Practical work
Secure coding	validation.py, domain.py, RedactingDecorator, ProviderConfig; run sections 5–6
Advanced OOP for complex problems	Frozen slotted value objects, ABCs, a runtime-checkable protocol, injected dependencies
Design patterns	All four classic patterns in one cohesive engine, each critiqued in Task 2
Large-scale architecture, robustness	build_risk_service root, documented pipeline ordering, degradation in run section 6
AI adaptability	model_registry.py, feature_store.py; run section 7 changes the live version with no code change

The Unit 6 artefact supplies the concurrency foundation the capstone reuses: FeatureStore takes a re-entrant lock specifically because compute_vector calls get internally, where a plain Lock would self-deadlock.

Task 2 – Showcase Artefacts (GitHub repository: [INSERT YOUR GITHUB REPOSITORY URL])

Repository: [INSERT YOUR GITHUB REPOSITORY URL]. Code is cited as artefact evidence rather than reproduced.

2.1 Artefact A — Thread-safe concurrent programming (Unit 6)

bank_account.py and transaction_simulator.py with 39 tests: an account mutated under a re-entrant lock, a transfer service, and a simulator driving concurrent transfers. **Principles:** encapsulated state behind methods, custom exception types for domain failure, and a re-

entrant lock so a guarded method calling another guarded method cannot deadlock. **Challenge:** the simulator's first succeeded/failed figures did not reconcile with its screenshot, teaching me to reproduce evidence rather than remember it. Because concurrency correctness can only be demonstrated, every lock in the capstone carries a test that fails without it.

2.2 Artefact B — Strategy: interchangeable scoring algorithms

RiskScoringStrategy defines score(features, model_ref) -> StrategyOutput, implemented three ways: RuleBasedScoringStrategy (weighted rules), WeightedStatisticalStrategy (logistic scoring) and ModelBackedScoringStrategy (through ScoringBackend); DecisionPolicy maps a normalised score onto ALLOW/REVIEW/BLOCK at 0.45 and 0.80. Because a fraud engine must run an explainable rule set in one environment and a learned model in another, StrategyOutput is uniform and contributions records which inputs drove the number, letting an auditor reconstruct a decision without the model. **Critical commentary.** Strategy forces every algorithm into (features) → StrategyOutput, so one rejecting a transaction outright must smuggle that through contributions or break the interface — a narrower contract chosen deliberately, since a richer return type would push policy into the algorithms.

2.3 Artefact C — Decorator: dynamic governance layers

ScoringComponent declares a name property and a score method and ScoringDecorator holds the wrapped component; five layers — RedactingDecorator, MetricsDecorator, AuditingDecorator, CachingDecorator, FallbackDecorator — are composed by compose_scoring_pipeline(), the single assembly point, so a client sees redacting(metered(audited(cached(rules-v1)))). **Critical commentary — a defect found and fixed.** Assembly originally placed AuditingDecorator *inside* FallbackDecorator. Every per-decorator test passed; the defect appeared only when a test asserted that a degraded decision still reaches the audit sink, which it did not, because the substituted decision was produced outside the auditor. The fix moves auditing outside the fallback, and the docstring records why.

2.4 Artefact D — Visitor: analytics and reporting

ReportVisitor declares visit_batch, visit_scored_transaction and result; four visitors — RiskSummaryVisitor, BlockedTransactionVisitor(minimum_score), JsonExportVisitor and ChannelRiskVisitor — run through a render(batch, visitor) driver. ScoredBatch and ScoredTransaction are immutable value objects that should not accumulate report logic, while reporting needs operations added repeatedly; adding a report is one class and no edit to the elements, demonstrated by ChannelRiskVisitor. **Critical commentary.** A new *element* type would force every visitor to change, which is acceptable because the element hierarchy is closed. The subtler risk is redaction drift: JsonExportVisitor exports only the last four characters of the account number, and the run asserts that no exported record contains a full identifier.

2.5 Artefact E — Abstract Factory: swapping AI service providers

AiProviderFactory creates a *family*: create_config(), create_backend() and create_explanation_generator(), with LocalHeuristicProviderFactory producing an on-premise heuristic backend and template explainer and HostedLlmProviderFactory a networked backend and narrative explainer, both around an **injected transport callable**. assemble() returns the family only after a consistency check, because a hosted endpoint with redaction off or a narrative explainer paired with a backend that cannot produce one are contradictions. **Critical commentary.** With two families this is arguable, but the family relationship is the

real invariant and a tuple-returning function would make the consistency check optional. The payoff is that nothing in strategies.py or service.py imports requests, httpx or a vendor SDK, so the remote path is exercised offline.

2.6 Artefacts F and G — Model Registry and Feature Store

Registry (model_registry.py). ModelRecord is immutable — name, version, stage, artifact reference, metrics, registered_by — and ModelRegistry is thread-safe, enforcing _ALLOWED_TRANSITIONS. Run section 7 shows the lifecycle: 2.1.0 live, 2.2.0 registered into DEVELOPMENT, promoted through STAGING to PRODUCTION, 2.1.0 archived and the next decision recording transaction-risk:2.2.0. **Defect found and fixed:** promote() returned the updated record but never wrote it back into the internal mapping; single-call tests passed because the return value was correct, and the bug surfaced only when a test promoted a version and then asked the registry what was live — an assertion on a return value cannot detect a missing state mutation. The registry now asserts through a second read path (get_production, history) after every transition.

Feature Store (feature_store.py). FeatureDefinition (name, version, description, extractor, owner) backs a store with a re-entrant lock and seven features; re-registering the same name and version is a no-op so start-up is idempotent, while a *different* version under an existing name raises. **Critical commentary.** The decisive choice was refusing to treat a missing feature as zero or a non-finite value silently: the rules engine skips a rule whose feature is absent, recording the omission by its absence from contributions, and compute converts a non-finite extractor result into FeatureStoreError.

2.7 Artefact H — Dependency Injection and inversion of control

RiskAssessmentService takes its component, feature store, feature names, context provider and explainer as constructor arguments. ServiceContainer is a deliberately minimal locator with explicit factories, lazy singleton caching and a loud ValidationError naming the missing key, and build_risk_service() is the composition root, the only function naming concrete implementations. Every dependency that would otherwise touch the outside world — transport, clock, sink, backend — is a constructor argument, which makes the suite deterministic and offline. **Critical commentary.** A framework container would be over-engineering at this size, so the container is about 45 lines and prefers explicit factories to convention-based resolution; a service locator hides dependencies from the type checker, so it stays at the outer edge (Fowler, 2004).

2.8 Artefact I — Testing: mocking and AI-assisted testing without live models

capstone/tests/ holds nine modules and 355 tests at 99% coverage of 2,512 statements, running in under 1.5 seconds entirely offline (394 tests including Unit 6). conftest.py names its doubles by category: **stubs** (CannedTransport, FailingBackend, StubScoringStrategy), a **fake** (RecordingBackend) and **spies** (CallRecorder); CannedTransport takes an optional payload *or* error, so three lines cover success and outage.

Reflection on AI-assisted testing. An AI assistant drafted tests from class contracts and found boundary cases I had not enumerated — a boolean score, which is an int in Python and would otherwise pass isinstance, and a payload missing the score key. It also created risk: confidently wrong assertions, including a test that searched for the letter "a" to assert a feature name was absent (explanation text contains one), a redaction expectation of one masked character when the implementation masks all but the last four, and a contract that

did not exist, that history() returns an empty list for an unknown model when it deliberately raises. None were detectable by reading the generated test, only by running it and understanding why it failed, so AI assistance is an accelerant for breadth, not a substitute for judgement about what a correct assertion means.

2.9 Capstone: scalable architecture, robustness and AI-readiness

The capstone is a **transaction risk-assessment engine**. *Scalability:* feature computation is pure and parallelisable, caching is an inner decorator replaceable by a distributed cache, the registry centralises lifecycle state, and provider families isolate network mechanics behind one callable. *Robustness and AI-readiness:* 355 deterministic tests assert degradation reaching the audit trail, redaction before any payload is seen, malformed replies rejected, promotion changing attribution, and illegal transitions refused; abstraction via ScoringBackend, contracts via StrategyOutput and versioned FeatureDefinition, and a registry that makes model version a runtime fact.

Figure 1 shows how these parts compose: the composition root, the decorator chain in its defended order, the strategy/backend seam, the adapters and the visitor.

Figure 1 — Capstone architecture: composition root, scoring pipeline and extension seams

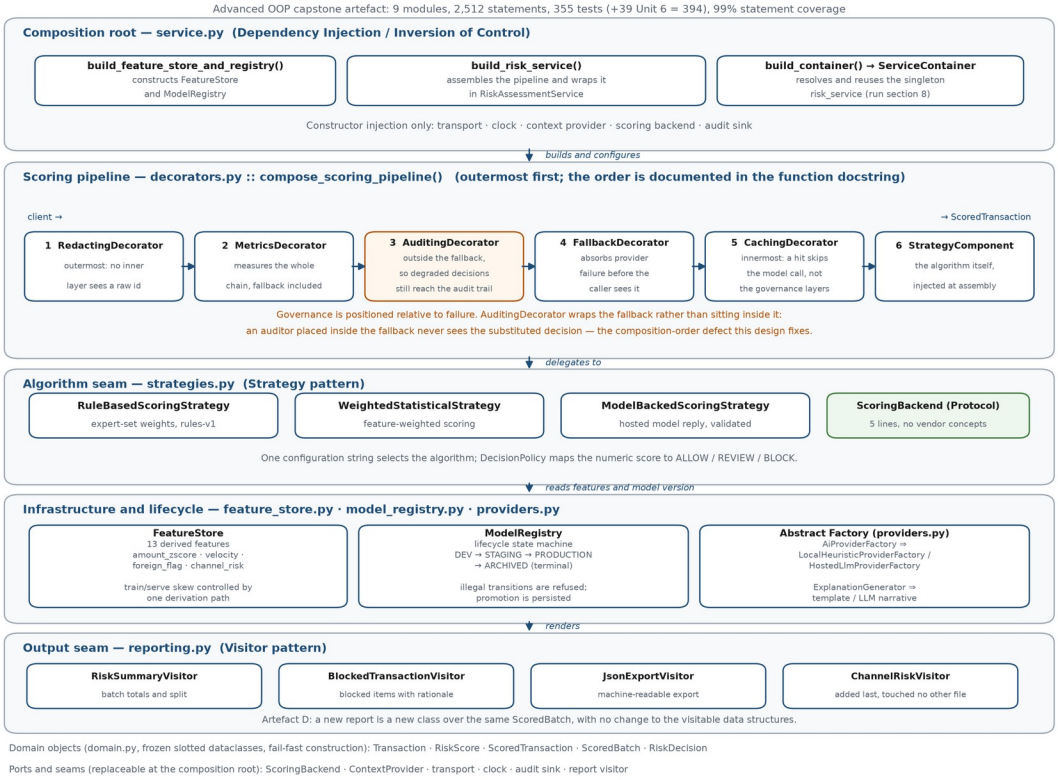


Figure 1 — Capstone architecture

Running-system evidence: evidence/demo_run_output.txt captures eight sections — a six-transaction batch yielding two allow, three review and one block; six audit records containing no identifier; masking before egress (GB33BUKB20201555555555 → **********5555); a provider outage with a three-attempt retry budget and explicit degradation; hostile-payload rejection; model promotion with lifecycle enforcement; and singleton resolution.

2.10 Cross-cutting critical commentary

Trade-offs. Every pattern was chosen against a rejected alternative and the rejections are part of the work: Strategy over a scorer hierarchy, Decorator over subclass combinations, Visitor over report methods, Abstract Factory over a factory function, constructor injection over a container inside the domain. **Testing complexity.** Both defects were cases where tests were individually correct and the system collectively wrong. **Abstraction versus over-engineering.** Every abstraction required a rejected alternative and a stated cost; the sixth decorator was rejected, and the minimal container is the seam for provider selection rather than a general-purpose framework.

Task 3 – Reflect on Skill Development

An e-Portfolio is a curated, purposeful collection of evidence rather than a chronological stack of exercises (Lorenzo and Ittelson, 2005), which is why every claim below is tied to a named artefact or capture.

3.1 Academic skills

Critical thinking. The formative experience was the composition-order defect. My first reaction was to distrust the test; the correct reaction was to distrust the design. I now ask what a passing test *cannot* see before asking whether it should pass, and applied to the registry defect that question produced my working rule about observing state through a second access path.

Ethical considerations in AI-driven OO design. *Bias propagation:* policy lives in data rather than a class hierarchy, so rule weights and thresholds are inspectable, versionable values that a reviewer can read directly. The residual risk is real: those weights are expert-set rather than learned, so they encode the blind spots of whoever set them — the engine makes bias *inspectable*, not *absent*. *Transparency:* `rationale` and `contributions` are contract fields, so an algorithm that cannot say which inputs drove its number cannot conform, and a provider's narrative is sanitised as untrusted content rather than treated as fact. *Avoiding black-box coupling:* `ScoringBackend` is five lines with no vendor concepts and no domain module imports a vendor SDK, so the coupling is a revocable composition-root decision. *Auditability:* every decision produces a structured record — component, model version, score, decision, rationale — asserted by the run to contain no account or customer identifier.

Problem-solving and refactoring. Refactoring was driven by measurable signals rather than taste: extracting `validation.py` removed duplication across five modules and made the security rules auditable in one file, and extracting `compose_scoring_pipeline()` collapsed four copies of an ordering decision into one documented function. I also resisted over-refactoring: a shared base class for the reporting visitors would create inheritance without polymorphism, since their `result()` shapes differ deliberately.

Research and self-study. Reading fell into three groups: pattern and principle sources (Gamma et al., 1994; Martin, 2000, 2017; Fowler, 2004, 2018) for vocabulary and named trade-offs; AI-systems sources (Sculley et al., 2015; Mitchell et al., 2019) for the feature-store and registry arguments; and risk and fairness sources (Aven and Thekdi, 2025; Barocas et al., 2023). The most useful outcome was negative: none of the pattern literature treats *governance* layers — audit, redaction, degradation — as first-class composition concerns, so the pipeline-order reasoning here came from analysing my own failure modes.

3.2 Employability skills

Time management. No single session could hold the whole assessment, so I worked in artefact-sized increments — one pattern, one test module, one evidence capture — so evidence capture happened alongside construction rather than being reconstructed at the end. My concrete failure was underestimating test-writing: closing coverage from 87% to 99% took three new modules and 143 tests and produced the two most valuable findings in the portfolio, which argues for budgeting test time as design time.

Communication and literacy. The clearest gain was writing docstrings that carry a decision rather than a description: `compose_scoring_pipeline` states the order and justifies each position, and `_ALLOWED_TRANSITIONS` documents why `ARCHIVED` is terminal. Writing the degradation marker as a machine-readable prefix (`[degraded: primary unavailable]`) was also a communication decision, since the same form is legible to an operator, a downstream consumer and a test.

Resilience and interpersonal skills. Resilience meant sitting with a failing test I believed was correct for longer than was comfortable, and changing the design instead of the assertion. On collaboration I must be explicit about the limits of my evidence: this portfolio is submitted as individual work, and Appendix B records the collaboration evidence I hold and what must be attached from personal records. Reviewing another student's decorator implementation nonetheless forced me to articulate my own ordering decision out loud.

Task 4 – Professional Development Plan (PDP)

4.1 Strengths and areas for improvement

Strengths (evidenced)	Areas for improvement (evidenced)
Nine modules with clear boundaries and one composition root naming concrete types	Documentation of intent is uneven — reasoning sits in docstrings, not one design record
Test discipline as a design tool: 355 offline tests at 99% coverage	Test-*shape* thinking arrived late; two defects escaped individually correct tests
Security as structure: immutable validated domain objects, boundary redaction, untrusted replies rejected	Data-side depth: fairness metrics and drift detection understood conceptually, not implemented
Registry and feature store implemented, not merely named	Operational thinking: retries, fallback and audit exist, but no monitoring or drift artefact
Honest scoping: abstractions rejected, residual risks documented	Early drafts were descriptive and had to be cut towards reasoning

4.2 Future goals and actions

Goal 1 — Deepen expertise in AI-aware software architecture. *Certification:* AWS Certified Machine Learning Engineer – Associate, which forces the operational vocabulary — monitoring, endpoint scaling, drift — that my design currently assumes someone else provides; Cloud Practitioner comes first. *Reading:* Sculley et al. (2015) with a design-review lens, Kleppmann (2017) for the distributed-systems reasoning a durable registry needs, and the MLflow (MLflow, 2026) and Feast (Feast, 2026) documentation as reference implementations to compare against my interfaces. *Applied project:* a persistent registry with an append-only promotion audit, and drift detection added as a **new visitor**, the cleanest demonstration that the Visitor's extension claim is real.

Goal 2 — Advanced testing strategies for intelligent systems. *Reading and practice:* property-based testing with Hypothesis (Hypothesis, 2026), because the defect classes I hit — missing state mutation, wrong composition order — are exactly what property-based and metamorphic testing target; a property such as "for every promotion, the registry's production version equals the last promoted version" would have caught the registry defect. *Skill:* contract testing for ScoringBackend, so every future provider is validated against one behavioural contract. *Applied project:* an evaluation harness comparing rules-v1 against a trained model on held-out data, reporting accuracy and fairness metrics per subgroup, to measure the residual risk of expert-set weights identified in Task 3.

Goal 3 — Ethical software engineering in AI-integrated applications. *Reading:* Barocas et al. (2023) properly rather than selectively; the NIST AI Risk Management Framework (NIST, 2023) as a governance structure for the audit and registry design; and the EU AI Act's high-risk obligations (European Parliament and Council, 2024), since credit scoring is a named high-risk category requiring auditability and documented model lifecycle. *Practice:* adopt model cards (Mitchell et al., 2019) as required artefacts for every registered version, extending ModelRecord with a required promotion rationale and refusing PRODUCTION promotion without it. *Applied project:* implement the fairness evaluation as a fifth reporting visitor, making bias a repeated measurement rather than a one-off conversation.

4.3 Reflective model applied — Gibbs' Reflective Cycle

Gibbs (1988) frames the event that produced the most learning: the composition-order defect. Kolb's (1984) cycle, the Johari Window (Luft and Ingham, 1955) and Bloom's revised taxonomy (Anderson and Krathwohl, 2001) were the alternatives; Gibbs was chosen because the trigger is one bounded failure whose cause can be traced.

1. Description. I had assembled the pipeline and written a test module per decorator; all passed. I then wrote a test asserting that when the hosted provider is unreachable, the degraded decision still reaches the audit trail. It failed: the sink recorded nothing for that transaction.

2. Feelings. First frustration and a suspicion the new test was wrong, because the per-decorator tests were green, giving way to unease on inspecting the assembly code, where the ordering was plainly visible. The discomfort marked the moment my mental model shifted from "a chain of layers" to "a chain of layers *with governance positioned relative to failure*".

3. Evaluation. The immediate cause was that AuditingDecorator wrapped the strategy before FallbackDecorator wrapped the result, so the substituted decision was produced outside the auditor. The deeper cause was that four places assembled the pipeline, making the ordering an implicit convention nobody could review; without the injectable sink the defect would have surfaced in production as an incident, not a design error.

4. Analysis. The Decorator pattern's strength — composable, independent layers — is also its characteristic hazard: correctness depends on ordering, and ordering is invisible to the type system because every decorator shares one type. Consolidating assembly into one documented function converts a hidden convention into a reviewable artefact. Mitchell et al. (2019) argue a model's intended use should be a first-class artefact; the equivalent here is that each layer's *position relative to failure* deserves the same explicitness.

5. Conclusion. Individually correct tests can sum to a collectively incorrect system, and that is a design smell rather than a testing gap. Two rules followed and now apply in this codebase: governance layers are positioned relative to failure modes with the positioning documented at the assembly point, and every stateful component has at least one test reading state back through a different path from the one it wrote through.

6. Action plan. Next project I will (i) write the assembly function before the individual layers, so the ordering is decided once and deliberately; (ii) write one integration test per *failure mode* rather than per class; and (iii) re-read every assembly point at review asking "what does this ordering make invisible?".

References

Anderson, L.W. and Krathwohl, D.R. (2001) *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. New York: Longman.

Aven, T. and Thekdi, S. (2025) *Risk Science: An Introduction*. 2nd edn. Abingdon: Routledge.

Barocas, S., Hardt, M. and Narayanan, A. (2023) *Fairness and Machine Learning: Limitations and Opportunities*. Cambridge, MA: MIT Press.

European Parliament and Council (2024) Regulation (EU) 2024/1689 of the European Parliament and of the Council laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). *Official Journal of the European Union*, L 1689. Available at: https://eur-lex.europa.eu/eli/reg/2024/1689/oj (Accessed: 17 October 2026).

Feast (2026) *Feast: Feature Store for Machine Learning*. Available at: https://docs.feast.dev/ (Accessed: 17 October 2026).

Fowler, M. (2004) *Inversion of Control Containers and the Dependency Injection Pattern*. Available at: https://martinfowler.com/articles/injection.html (Accessed: 17 October 2026).

Fowler, M. (2018) *Refactoring: Improving the Design of Existing Code*. 2nd edn. Boston, MA: Addison-Wesley.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston, MA: Addison-Wesley.

Gibbs, G. (1988) *Learning by Doing: A Guide to Teaching and Learning Methods*. Oxford: Further Education Unit, Oxford Polytechnic.

Hypothesis (2026) *Hypothesis: Property-Based Testing for Python*. Available at: https://hypothesis.readthedocs.io/ (Accessed: 17 October 2026).

Kleppmann, M. (2017) *Designing Data-Intensive Applications*. Sebastopol, CA: O'Reilly Media.

Kolb, D.A. (1984) *Experiential Learning: Experience as the Source of Learning and Development*. Englewood Cliffs, NJ: Prentice Hall.

Lorenzo, G. and Ittelson, J. (2005) *An Overview of E-Portfolios*. Boulder, CO: Educause Learning Initiative.

Luft, J. and Ingham, H. (1955) 'The Johari window: a graphic model of interpersonal awareness', *Proceedings of the Western Training Laboratory in Group Development*. Los Angeles: University of California.

Martin, R.C. (2000) *Design Principles and Design Patterns*. Available at: https://web.archive.org/web/20150906155800/http://www.objectmentor.com/resources/articles/Principles_and_Patterns.pdf (Accessed: 17 October 2026).

Martin, R.C. (2017) *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston, MA: Prentice Hall.

Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I.D. and Gebru, T. (2019) 'Model cards for model reporting', *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT* '19)*, pp. 220–229. doi:10.1145/3287560.3287596.

MLflow (2026) *MLflow Model Registry*. Available at: https://mlflow.org/docs/latest/model-registry.html (Accessed: 17 October 2026).

National Institute of Standards and Technology (NIST) (2023) *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. NIST AI 100-1. Gaithersburg, MD: NIST.

OWASP Foundation (2021) *OWASP Top 10:2021 – The Ten Most Critical Web Application Security Risks*. Available at: https://owasp.org/Top10/ (Accessed: 17 October 2026).

Python Software Foundation (2026) *unittest.mock — mock object library*. Available at: https://docs.python.org/3/library/unittest.mock.html (Accessed: 17 October 2026).

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F. and Dennison, D. (2015) 'Hidden technical debt in machine learning systems', *Advances in Neural Information Processing Systems 28 (NIPS 2015)*, pp. 2503–2511.

Appendix A — Artefact and evidence index

#	Artefact	File(s)	Evidence	Task 2
A	Thread-safe coding exercise (Unit 6)	unit6/ bank_account.py, unit6/transaction_ simulator.py, unit6/test_banking _system.py	39 tests passing; simulator run output	2.1
B	Strategy pattern	capstone/ strategies.py	test_strategies.py; run section 2	2.2
C	Decorator pattern	capstone/ decorators.py	test_decorators.py ; run section 6a	2.3
D	Visitor pattern	capstone/ reporting.py	test_reporting.py; run section 3	2.4
E	Abstract Factory	capstone/ providers.py	test_providers.py; run sections 6b–6c	2.5
F	Model Registry	capstone/ model_registry.py	test_model_registr y.py; run section 7	2.6
G	Feature Store	capstone/ feature_store.py	test_feature_store. py; run section 2	2.6
H	DI / IoC	capstone/ service.py	test_service.py; run section 8	2.7
I	Mocking / AI-driven testing	capstone/tests/ conftest.py + nine test modules	evidence/ test_run_output.txt ; evidence/coverag e_report.txt	2.8
J	Secure validation layer	capstone/ validation.py, capstone/domain. py	test_validation.py, test_domain.py; run section 5	1.1
K	Runtime demonstration	demo_run.py	evidence/ demo_run_output. txt (8 sections)	2.9
L	Coverage report	—	evidence/ coverage_html/ index.html	2.8

Appendix B — Collaboration and integrity evidence (student to complete)

Twenty per cent of the module mark depends on evidence of independent working, or of individual contribution to a group project, and that evidence must be the student's own real trace. Nothing here has been generated on the student's behalf; the table is a checklist to complete, using the summary in integrity_and_collaboration_notes.md and the log in contribution_log_template.md.

Evidence type	What to attach	Held?
Repository contribution history	Git log with your commits, dates and messages, or git shortlog -sne output	Yes / No
Task ownership	Task board export (Trello/Jira/GitHub Projects) showing items you owned and their state	Yes / No
Meeting records	Dated minutes or notes with your recorded actions	Yes / No
Peer review	Comments you gave and received on another student's artefact	Yes / No
Tutor feedback	Feedback on drafts, with your response to it	Yes / No
Independent working	Dated study log or signed planner extract showing your own schedule	Yes / No

Appendix C — Honest statement of limits

1. **The default rule weights are expert judgement, not evidence.** The weights and the elevated-category set are plausible values, not values derived from data. They are versioned and inspectable, but inspectable is not validated, and the fairness risk described in Task 3 lives here.

2. **The model-backed path is exercised against test doubles, never a real model.** This is deliberate — no API keys, no cost, no network — and it validates the integration contract thoroughly. It does not validate that any real model answers correctly; the hosted transport is the seam where that would change.

3. **The registry and feature store are in-memory.** They demonstrate lifecycle and versioning semantics with concurrency safety, but durability, migration and multi-process consistency are out of scope, which is why Goal 1 names a persisted registry as the next applied project.